

GPU 工作站安装与验证教程

Ubuntu 22.04 + RTX 4090 + CUDA / PyTorch / Docker

服务器	192.168.1.46
系统用户	ubuntu
生成日期	2026-05-26
适用对象	第一次接触 Linux / CUDA / Docker 的使用者
文档目标	说明每个软件类别的安装命令、验证命令和判断是否成功的方法

阅读方式

如果环境已经安装好，不需要重复执行安装命令；只执行验证命令即可。安装命令用于以后重装或对照排查。

目录

1. 登录服务器与基础检查
2. APT 源和系统工具
3. NVIDIA Driver
4. CUDA Toolkit 12.9
5. cuDNN / NCCL / TensorRT
6. Docker / Compose / NVIDIA Container Toolkit
7. Miniconda 与 dl-general 环境
8. PyTorch GPU 与 JupyterLab
9. CUDA Samples 编译与验证
10. 一键综合验证脚本
11. 本次未安装项目和安全建议

1. 登录服务器与基础检查

先从你的 Windows 电脑 CMD 或 PowerShell 登录服务器。登录成功后，提示符一般类似 ubuntu@ubuntu:~\$。

```
ssh ubuntu@192.168.1.46
```

基础检查命令

```
lsb_release -a
uname -a
whoami
id
lscpu | head -20
free -h
lsblk -o NAME,SIZE,TYPE,MOUNTPOINTS
```

- lsb_release 能看到 Ubuntu 22.04.5 LTS。
- whoami 输出 ubuntu。
- free -h 能看到约 123 GiB 内存。
- lsblk 能看到约 1.8 TiB NVMe 磁盘。

2. APT 源和系统工具

用途

提供下载安装软件需要的基础工具、编译工具、压缩工具、终端工具、科学计算基础库。

安装命令

```
sudo apt-get update
sudo apt-get install -y \
  openssh-server curl wget ca-certificates gnupg lsb-release software-properties-
common \
  git git-lfs vim htop net-tools unzip zip rsync tree tmux nano nvidia \
  build-essential gcc g++ make cmake ninja-build gdb gfortran pkg-config \
  python3-pip python3-dev python3-venv \
  ffmpeg mesa-utils \
  openmpi-bin libopenmpi-dev mpich libmpich-dev \
  libopenblas-dev liblapack-dev libatlas-base-dev libscalapack-openmpi-dev \
  libfftw3-dev libeigen3-dev
git lfs install
```

验证命令

```
ssh -V
git --version
git lfs version
gcc --version | head -1
g++ --version | head -1
gfortran --version | head -1
make --version | head -1
cmake --version | head -1
ninja --version
mpirun --version | head -1
ldconfig -p | grep blas | head
ldconfig -p | grep lapack | head
ldconfig -p | grep fftw | head
```

通过标准

- 每条命令都能输出版本或库信息。
- 没有出现 command not found。
- Git、GCC、CMake、Ninja、MPI、BLAS/LAPACK/FFTW 都能查到。

注意事项

这些是后面安装 CUDA、编译样例、安装 Python 包时经常会用到的基础依赖。

3. NVIDIA Driver

用途

让 Ubuntu 正确识别 RTX 4090 显卡，并为 CUDA、PyTorch、Docker GPU 容器提供底层驱动。

安装命令

```
#  
#  
sudo apt-get update  
ubuntu-drivers devices  
sudo ubuntu-drivers autoinstall  
sudo reboot
```

验证命令

```
nvidia-smi  
nvidia-smi -q | head -80  
lspci | grep -Ei 'nvidia|vga|3d|display'
```

通过标准

- nvidia-smi 能正常打开，不报错。
- 显卡显示 NVIDIA GeForce RTX 4090。
- Driver Version 显示 580.159.03。

注意事项

显卡驱动属于高风险组件。今天安装 CUDA 时没有安装会替换当前 580 驱动的总元包，而是保留了当前可用驱动。

4. CUDA Toolkit 12.9

用途

提供 nvcc 编译器、CUDA 命令行工具和 CUDA 开发库，用于编译 CUDA 程序和支撑本地开发。

安装命令

```
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt-get update

#          580          CUDA
sudo apt-get install -y \
    cuda-compiler-12.9 \
    cuda-command-line-tools-12.9 \
    cuda-libraries-dev-12.9

echo 'export PATH=/usr/local/cuda-12.9/bin:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=/usr/local/cuda-12.9/lib64:${LD_LIBRARY_PATH}' >> ~/.bashrc
bashrc
```

验证命令

```
export PATH=/usr/local/cuda-12.9/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-12.9/lib64:${LD_LIBRARY_PATH:-}
nvcc --version
which nvcc
ls -ld /usr/local/cuda /usr/local/cuda-12.9
```

通过标准

- nvcc 输出 release 12.9, V12.9.86。
- which nvcc 输出 /usr/local/cuda-12.9/bin/nvcc。
- /usr/local/cuda 指向 /usr/local/cuda-12.9。

注意事项

nvidia-smi 里显示的 CUDA Version 来自驱动能力；nvcc --version 显示的是实际安装的 CUDA Toolkit 版本。

5. cuDNN / NCCL / TensorRT

用途

cuDNN 用于深度学习算子加速，NCCL 用于多 GPU 通信，TensorRT 用于模型推理优化和部署。

安装命令

```
sudo apt-get update
sudo apt-get install -y \
    cudnn9-cuda-12 \
    libnccl2 libnccl-dev \
    tensorrt python3-libnvinfer-dev
```

验证命令

```
ldconfig -p | grep cudnn | head
ldconfig -p | grep nccl | head
dpkg -l | grep -E 'cudnn|nccl|tensorrt|nvinfer'
which trtexec
python3 - <<'PY'
import tensorrt as trt
print('TensorRT:', trt.__version__)
PY
```

通过标准

- cuDNN 包显示 9.22.0.52。
- NCCL 包显示 2.30.4-1+cuda12.9。
- TensorRT 输出 10.16.1.11。
- which trtexec 输出 /usr/bin/trtexec。

注意事项

trtexec 没有模型时一般用 --help 或 which 验证即可，不需要强行运行模型转换。

6. Docker / Compose / NVIDIA Container Toolkit

用途

Docker 用来运行容器；NVIDIA Container Toolkit 让容器可以调用 RTX 4090 显卡。

安装命令

```
sudo apt-get update
sudo apt-get install -y docker.io docker-compose docker-compose-v2
sudo systemctl enable --now docker
sudo usermod -aG docker ubuntu

curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
| sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-
toolkit.list \
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-
keyring.gpg] https://#g' \
| sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

验证命令

```
docker --version
docker-compose --version
docker compose version
groups
sudo docker info --format '{{.ServerVersion}} {{json .Runtimes}}'
nvidia-container-cli info | head -80

# Docker Hub
sudo docker run --rm --gpus all nvidia/cuda:12.9.1-base-ubuntu22.04 nvidia-smi
```

通过标准

- ❑ `docker --version` 显示 Docker version 29.1.3。
- ❑ `docker compose version` 显示 Docker Compose version 2.40.3。
- ❑ `groups` 里能看到 `docker`。
- ❑ `docker info` 的 `Runtimes` 里能看到 `nvidia`。
- ❑ `nvidia-container-cli info` 能看到 NVIDIA GeForce RTX 4090。

注意事项

Docker GPU 容器测试需要访问 Docker Hub。今天本地 runtime 已验证正常，但拉取 `nvidia/cuda` 镜像时可能受网络影响超时。

7. Miniconda 与 dl-general 环境

用途

用 Conda 隔离 Python 深度学习环境，避免不同项目之间互相污染。

安装命令

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O ~/miniconda.sh
bash ~/miniconda.sh -b -p ~/miniconda3
source ~/miniconda3/etc/profile.d/conda.sh
conda init bash

conda create -y -n dl-general python=3.10
conda activate dl-general

pip install -i https://pypi.tuna.tsinghua.edu.cn/simple \
    numpy scipy pandas matplotlib opencv-python pyyaml tqdm rich \
    jupyterlab notebook ipykernel tensorboard wandb \
    scikit-learn scikit-image h5py pillow einops hydra-core omegaconf

python -m ipykernel install --user --name dl-general --display-name "Python (dl-general)"
conda env export > ~/workspace/dl-general.yml
```

验证命令

```
source ~/miniconda3/etc/profile.d/conda.sh
conda --version
conda env list
conda activate dl-general
python --version
pip list | grep -E 'numpy|scipy|pandas|jupyterlab|opencv|scikit|tensorboard'
jupyter lab --version
jupyter kernelspec list
```

通过标准

- ❑ `conda env list` 里能看到 `dl-general`。
- ❑ `python --version` 能看到 Python 3.10。
- ❑ `jupyter lab --version` 能输出版本。
- ❑ `jupyter kernelspec list` 里能看到 Python (dl-general)。

注意事项

以后做项目时建议优先新建独立 Conda 环境；dl-general 适合做通用 GPU 测试和基础实验。

8. PyTorch GPU 与 JupyterLab

用途

PyTorch 是主要深度学习框架；JupyterLab 用于浏览器中运行 notebook。

安装命令

```
source ~/miniconda3/etc/profile.d/conda.sh
conda activate dl-general

pip install --index-url https://download.pytorch.org/whl/cu128 \
    torch torchvision torchaudio
```

验证命令

```
source ~/miniconda3/etc/profile.d/conda.sh
conda activate dl-general
python - <<'PY'
import torch
print('torch:', torch.__version__)
print('torch cuda:', torch.version.cuda)
print('cuda available:', torch.cuda.is_available())
if torch.cuda.is_available():
    print('gpu:', torch.cuda.get_device_name(0))
    print('vram_gb:', round(torch.cuda.get_device_properties(0).total_memory /
1024**3, 2))
    x = torch.randn(1024, 1024, device='cuda')
    print('cuda tensor test:', float((x @ x).sum().detach().cpu()))
PY

jupyter lab --ip=0.0.0.0 --port=8888 --no-browser
```

通过标准

- ☐ torch 显示 2.11.0+cu128。
- ☐ torch cuda 显示 12.8。
- ☐ cuda available 显示 True。
- ☐ gpu 显示 NVIDIA GeForce RTX 4090。
- ☐ cuda tensor test 能输出数字并且不报错。

注意事项

PyTorch 的 cu128 wheel 自带 CUDA 12.8 运行库，和系统 CUDA 12.9 可以共存。系统 CUDA 主要用于 nvcc 编译和系统开发库。

9. CUDA Samples 编译与验证

用途

CUDA Samples 用实际 CUDA 程序验证 nvcc、CUDA runtime 和显卡是否能正常编译运行。

安装命令

```
cd /home/ubuntu/cuda-samples

#          build      root
sudo chown -R ubuntu:ubuntu /home/ubuntu/cuda-samples
rm -rf build

# RTX 4090    sm_89      CMakeLists.txt    CUDA 12.9
#      CMAKE_CUDA_ARCHITECTURES      89
grep -R "CMAKE_CUDA_ARCHITECTURES" -n CMakeLists.txt cpp | head

cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=89 \
  | tee ~/workspace/cuda-samples-configure.log

cmake --build build --target deviceQuery vectorAdd p2pBandwidthLatencyTest -j$(nproc) \
  | tee ~/workspace/cuda-samples-build.log
```

验证命令

```
cd /home/ubuntu/cuda-samples
./build/cpp/1_Uutilities/deviceQuery/deviceQuery
./build/cpp/0_Introduction/vectorAdd/vectorAdd
./build/cpp/5_Domain_Specific/p2pBandwidthLatencyTest/p2pBandwidthLatencyTest

ls -lh \
  build/cpp/1_Uutilities/deviceQuery/deviceQuery \
  build/cpp/0_Introduction/vectorAdd/vectorAdd \
  build/cpp/5_Domain_Specific/p2pBandwidthLatencyTest/p2pBandwidthLatencyTest
```

通过标准

- ☐ deviceQuery 最后显示 Result = PASS。
- ☐ vectorAdd 最后显示 Test PASSED。
- ☐ p2pBandwidthLatencyTest 能输出带宽和延迟矩阵。
- ☐ 三个二进制文件都存在并可执行。

注意事项

今天已经编译并验证通过。相关日志保存在 ~/workspace/cuda-samples-*.log。

10. 一键综合验证脚本

如果你不想逐项复制命令，可以把下面整段复制到服务器终端执行。执行后会生成 ~/workspace/VERIFY_RESULT.txt。

```

cat > /tmp/verify_gpu_env.sh <<'SCRIPT_END'
#!/usr/bin/env bash
set -e
export PATH=/usr/local/cuda-12.9/bin:$HOME/miniconda3/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-12.9/lib64:${LD_LIBRARY_PATH:-}

echo "===== OS / User ====="
lsb_release -a
uname -a
whoami

echo "===== NVIDIA Driver ====="
nvidia-smi

echo "===== CUDA Toolkit ====="
nvcc --version
which nvcc

echo "===== CUDA Samples ====="
cd /home/ubuntu/cuda-samples
./build/cpp/1_Uutilities/deviceQuery/deviceQuery | tail -20
./build/cpp/0_Introduction/vectorAdd/vectorAdd

echo "===== cuDNN / NCCL / TensorRT ====="
ldconfig -p | grep cudnn | head || true
ldconfig -p | grep nccl | head || true
python3 - <<'PY'
import tensorrt as trt
print('System Python TensorRT:', trt.__version__)
PY

echo "===== Docker GPU Runtime ====="
docker --version
docker compose version || true
sudo docker info --format '{{.ServerVersion}}' '{{json .Runtimes}}'
nvidia-container-cli info | head -80

echo "===== Conda / PyTorch ====="
source ~/miniconda3/etc/profile.d/conda.sh
conda activate dl-general
python - <<'PY'
import torch
print('torch:', torch.__version__)
print('torch cuda:', torch.version.cuda)
print('cuda available:', torch.cuda.is_available())
if torch.cuda.is_available():
    print('gpu:', torch.cuda.get_device_name(0))
    x = torch.randn(1024, 1024, device='cuda')
    print('cuda tensor test:', float((x @ x).sum().detach().cpu()))
PY

echo "===== DONE ====="
SCRIPT_END
chmod +x /tmp/verify_gpu_env.sh
/tmp/verify_gpu_env.sh | tee ~/workspace/VERIFY_RESULT.txt

```

11. 单个软件安装与验证速查表

下面是按单个软件/组件整理的速查表。重装时可以参考“安装命令”；平时只需要执行“验证命令”。

软件	安装命令	验证命令	通过标准
OpenSSH Server	<code>sudo apt-get install -y openssh-server</code>	<code>ssh -V</code>	输出 OpenSSH 版本
curl	<code>sudo apt-get install -y curl</code>	<code>curl --version head -1</code>	输出 curl 版本
wget	<code>sudo apt-get install -y wget</code>	<code>wget --version head -1</code>	输出 wget 版本
Git	<code>sudo apt-get install -y git</code>	<code>git --version</code>	输出 Git 版本

软件	安装命令	验证命令	通过标准
Git LFS	<code>sudo apt-get install -y git-lfs && git lfs install</code>	<code>git lfs version</code>	输出 Git LFS 版本
vim	<code>sudo apt-get install -y vim</code>	<code>vim --version head -1</code>	输出 Vim 版本
htop	<code>sudo apt-get install -y htop</code>	<code>htop --version</code>	输出 htop 版本
nvidia-smi	<code>sudo apt-get install -y nvidia-smi</code>	<code>nvidia-smi</code>	能打开或输出版本
tmux	<code>sudo apt-get install -y tmux</code>	<code>tmux -V</code>	输出 tmux 版本
tree	<code>sudo apt-get install -y tree</code>	<code>tree --version</code>	输出 tree 版本
ffmpeg	<code>sudo apt-get install -y ffmpeg</code>	<code>ffmpeg -version head -1</code>	输出 ffmpeg 版本
mesa-utils	<code>sudo apt-get install -y mesa-utils</code>	<code>glxinfo -B head</code>	能输出 OpenGL 信息
GCC	<code>sudo apt-get install -y gcc</code>	<code>gcc --version head -1</code>	输出 gcc 版本
G++	<code>sudo apt-get install -y g++</code>	<code>g++ --version head -1</code>	输出 g++ 版本
GFortran	<code>sudo apt-get install -y gfortran</code>	<code>gfortran --version head -1</code>	输出 gfortran 版本
Make	<code>sudo apt-get install -y make</code>	<code>make --version head -1</code>	输出 make 版本
CMake	<code>sudo apt-get install -y cmake</code>	<code>cmake --version head -1</code>	输出 cmake 版本
Ninja	<code>sudo apt-get install -y ninja-build</code>	<code>ninja --version</code>	输出 Ninja 版本
OpenMPI	<code>sudo apt-get install -y openmpi-bin libopenmpi-dev</code>	<code>mpirun --version head -1</code>	输出 OpenMPI 版本
MPICH	<code>sudo apt-get install -y mpich libmpich-dev</code>	<code>mpichversion head</code>	输出 MPICH 信息
BLAS/OpenBLAS	<code>sudo apt-get install -y libopenblas-dev libatlas-base-dev</code>	<code>ldconfig -p grep blas head</code>	能查到 blas/openblas 库
LAPACK	<code>sudo apt-get install -y liblapack-dev</code>	<code>ldconfig -p grep lapack head</code>	能查到 lapack 库
SCALAPACK	<code>sudo apt-get install -y libscalapack-openmpi-dev</code>	<code>ldconfig -p grep scalapack head</code>	能查到 scalapack 库
FFTW	<code>sudo apt-get install -y libfftw3-dev</code>	<code>ldconfig -p grep fftw head</code>	能查到 fftw 库
Eigen	<code>sudo apt-get install -y libeigen3-dev</code>	<code>ls /usr/include/eigen3/Eigen head</code>	能列出 Eigen 头文件
NVIDIA Driver	<code>ubuntu-drivers autoinstall</code>	<code>nvidia-smi</code>	显示 RTX 4090 和 580.159.03
CUDA Compiler	<code>sudo apt-get install -y cuda-compiler-12.9</code>	<code>nvcc --version</code>	显示 release 12.9
CUDA Command Tools	<code>sudo apt-get install -y cuda-command-line-tools-12.9</code>	<code>ls /usr/local/cuda-12.9/bin head</code>	能看到 CUDA 工具
CUDA Dev Libraries	<code>sudo apt-get install -y cuda-libraries-dev-12.9</code>	<code>ls /usr/local/cuda-12.9/lib64 head</code>	能看到 CUDA 库
cuDNN	<code>sudo apt-get install -y cudnn9-cuda-12</code>	<code>ldconfig -p grep cudnn head</code>	能查到 cudnn 库
NCCL	<code>sudo apt-get install -y libnccl2 libnccl-dev</code>	<code>ldconfig -p grep nccl head</code>	能查到 nccl 库
TensorRT	<code>sudo apt-get install -y tensorrt python3-libnvinfer-dev</code>	<code>python3 -c "import tensorrt as trt; print(trt.__version__)"</code>	输出 10.16.1.11
Docker Engine	<code>sudo apt-get install -y docker.io</code>	<code>docker --version</code>	输出 29.1.3
Docker Compose v1	<code>sudo apt-get install -y docker-compose</code>	<code>docker-compose --version</code>	输出 1.29.2
Docker Compose v2	<code>sudo apt-get install -y docker-compose-v2</code>	<code>docker compose version</code>	输出 2.40.3

软件	安装命令	验证命令	通过标准
NVIDIA Container Toolkit	<code>sudo apt-get install -y nvidia-container-toolkit</code>	<code>nvidia-container-cli info head -80</code>	能看到 RTX 4090
Miniconda	<code>bash ~/miniconda.sh -b -p ~/miniconda3</code>	<code>source ~/miniconda3/etc/profile.d/conda.sh && conda --version</code>	输出 conda 版本
dl-general 环境	<code>conda create -y -n dl-general python=3.10</code>	<code>conda env list grep dl-general</code>	能看到 dl-general
JupyterLab	<code>pip install jupyterlab notebook ipykernel</code>	<code>jupyter lab --version</code>	输出 JupyterLab 版本
TensorBoard	<code>pip install tensorboard</code>	<code>tensorboard --version</code>	输出 TensorBoard 版本
PyTorch	<code>pip install --index-url https://download.pytorch.org/whl/cu128 torch</code>	<code>python -c "import torch; print(torch.__version__)"</code>	输出 2.11.0+cu128
torchvision	<code>pip install --index-url https://download.pytorch.org/whl/cu128 torchvision</code>	<code>python -c "import torchvision; print(torchvision.__version__)"</code>	输出 torchvision 版本
torchaudio	<code>pip install --index-url https://download.pytorch.org/whl/cu128 torchaudio</code>	<code>python -c "import torchaudio; print(torchvision.__version__)"</code>	输出 torchaudio 版本
PyTorch GPU	PyTorch	<code>python -c "import torch; print(torch.cuda.is_available())"</code>	输出 True
NumPy/SciPy/Pandas	<code>pip install numpy scipy pandas</code>	<code>python -c "import numpy, scipy, pandas; print('OK')"</code>	输出 OK
OpenCV	<code>pip install opencv-python</code>	<code>python -c "import cv2; print(cv2.__version__)"</code>	输出 OpenCV 版本
scikit-learn	<code>pip install scikit-learn</code>	<code>python -c "import sklearn; print(sklearn.__version__)"</code>	输出 sklearn 版本
scikit-image	<code>pip install scikit-image</code>	<code>python -c "import skimage; print(skimage.__version__)"</code>	输出 skimage 版本
h5py/Pillow/einops	<code>pip install h5py pillow einops</code>	<code>python -c "import h5py, PIL, einops; print('OK')"</code>	输出 OK
Hydra/OmegaConf	<code>pip install hydra-core omegaconf</code>	<code>python -c "import hydra, omegaconf; print('OK')"</code>	输出 OK
CUDA Samples	<code>cmake -S . -B build -DCMAKE_CUDA_ARCHITECTURES=89 && cmake --build build -j\$(nproc)</code>	<code>./build/cpp/0_Introduction/vectorAdd/vectorAdd</code>	显示 Test PASSED

12. 本次未安装项目和安全建议

本次未安装

- ☐ Isaac Sim / Isaac Lab
- ☐ ROS1 / ROS2
- ☐ MuJoCo / Gazebo / SAPIEN
- ☐ NVIDIA DIGITS
- ☐ Caffe / MXNet / Theano
- ☐ Eclipse / Qt4 / Java / Tcl-Tk

原因

这些软件通常要与具体项目、Python 版本、CUDA 版本严格匹配。直接全局安装容易冲突，后续建议按项目 README 单独创建 Conda 环境或 Docker 容器。

安全建议

安装完成并确认环境没问题后，建议修改服务器 SSH 密码，后续最好改用 SSH key 登录。

```
passwd
```